

OpenID AuthZEN

Yan Vitor Pires Pereira
yan.pp2006@aluno.ifsc.edu.br

30 de abril de 2026

Sumário

1	Introdução	1
2	Controle de Acesso e Arquitetura Baseada em Política	2
2.1	Evolução do controle de acesso	2
2.2	Arquitetura de referência para autorização	4
2.3	Gerenciamento de acesso baseado em política	4
2.4	Validações de acesso em tempo real	5
3	Protocolo AuthZEN	5
3.1	Exemplo de solicitação de acesso	6
3.2	Defesa em profundidade	6
4	Aplicação no contexto de IA Agêntica	7
4.1	Velocidade e volume de requisições	7
4.2	Tokens estáticos	7
4.3	Auditoria e rastreabilidade	7
5	Conclusão	8

1 Introdução

isso é autenticação e não controle de acesso

O controle de acesso está amplamente presente no mundo digital, como login com usuário e senha. No passado, aplicações de terceiros costumavam solicitar as credenciais do usuário (usuário e senha), se passando pelo mesmo. Porém, este método se provou perigoso. Esse modelo permitia acesso irrestrito, impossibilitando limitar o que a aplicação poderia fazer e dificultando a auditoria nos registros sobre quem de fato realizou a ação.

Para resolver este problema, protocolos de autorização e delegação como o OpenID Connect (OIDC) e o Security Assertion Markup Language (SAML) foram criados, mais conhecidos hoje como “login social” ou “single sign-on”. Eles permitiram a descentralização do login, comprovando de forma segura que o usuário é quem ele diz ser, sem precisar fornecer a senha que garante acesso total para terceiros.

As permissões fornecidas ao serviço terceirizado são geradas no momento da autenticação, precisando vincular todas as autorizações necessárias quando o login é realizado. Omri Gazzit cita como exemplo: “e se o Google

tentasse utilizar o OAuth 2.0 e o OpenID Connect para resolver a autorização de alta granularidade dos arquivos do Google Drive e Docs? [...] a aplicação precisaria descobrir quais documentos o usuário tem acesso no momento do login, podendo ser milhares, e vincular todos eles no token de acesso para liberar o uso” [3]. Esta fala aponta para dois possíveis problemas:

- Ultrapassar limite do HTTP: Se o sistema criar uma permissão para cada documento e colocar todas elas dentro de um token, o JWT (Json Web Token) ficaria gigantesco, excedendo os limites do protocolo HTTP.
- Permissões Obsoletas (Stale Tokens): Uma vez que o token é gerado, ele permanece válido até expirar. Se o acesso do agente for revogado antes dessa expiração, o token ainda continuará funcionando até o último momento, podendo causar brechas de segurança.

Se a personificação e os tokens estáticos já eram problemas para aplicações comuns, com o ritmo acelerado e volume de solicitações da inteligência artificial agêntica isso se tornou um risco ainda maior de segurança.

A fim de propor soluções para estes problemas, este relatório visa explicar a evolução dos modelos de controle de acesso, detalhar o funcionamento e as camadas de defesa do protocolo AuthZEN e demonstrar como esta arquitetura é fundamental para a viabilidade e segurança da IA Agêntica.

2 Controle de Acesso e Arquitetura Baseada em Política

2.1 Evolução do controle de acesso

Com o passar do tempo, se tornou necessário que os modelos de controle de acesso mudassem sua abordagem. Na década de 1980 surgiram as listas de controle de acesso (ACL), Unix e NT CACL por exemplo, elas respondem uma simples pergunta: "O usuário tem permissão para acessar este arquivo?". Porém, este modelo não se provou prático quanto à escalabilidade. É viável quando são apenas alguns arquivos e pessoas, mas e quando são milhares? Esse problema levou à criação do controle de acesso baseado em papéis (RBAC) nos anos 1990, ele tornou possível administrar as permissões dos usuários apenas modificando os papéis aos quais eles estão atrelados, todos de uma vez.

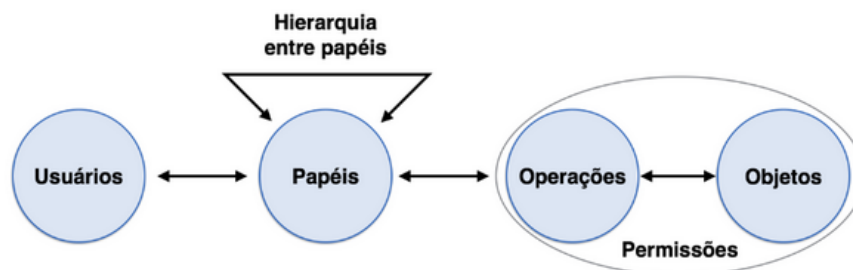


Figura 1: Modelo RBAC.

Fonte: Minicurso SBSeg [2]

Outro problema foi observado: quando um professor pode realizar impressões coloridas apenas se estiver no câmpus em seu horário de turno, por exemplo. Um papel novo teria que ser criado para atrelar somente àquele professor.

Para resolver este problema, por volta dos anos 2000 foi criado o controle de acesso baseado em atributos, que possibilitou a adição de diversas condições como cargo, expediente, localização geográfica, entre outros para filtrar ainda melhor a distribuição de privilégios.

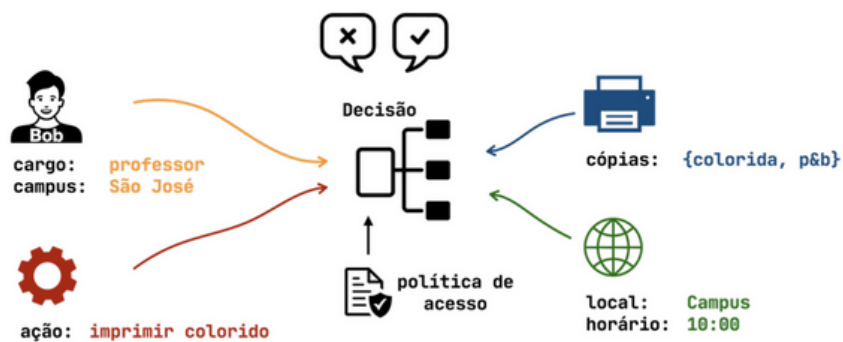


Figura 2: Modelo ABAC.
Fonte: Autorização e Controle de Acesso. [1]

Por volta do ano de 2020 se popularizou o controle de acesso baseado em relacionamentos com o Zanzibar da Google. Esse modelo é baseado em grafos (*graph-based*) e analisa as conexões entre os diversos usuários, grupos e recursos no sistema e as permissões atreladas a cada um deles com a finalidade de buscar uma linha que ligue o sujeito com a ação que ele quer realizar: "se o Usuário A é membro do Grupo B, e o Grupo B é proprietário da Pasta C, então o Usuário A tem acesso à Pasta C".

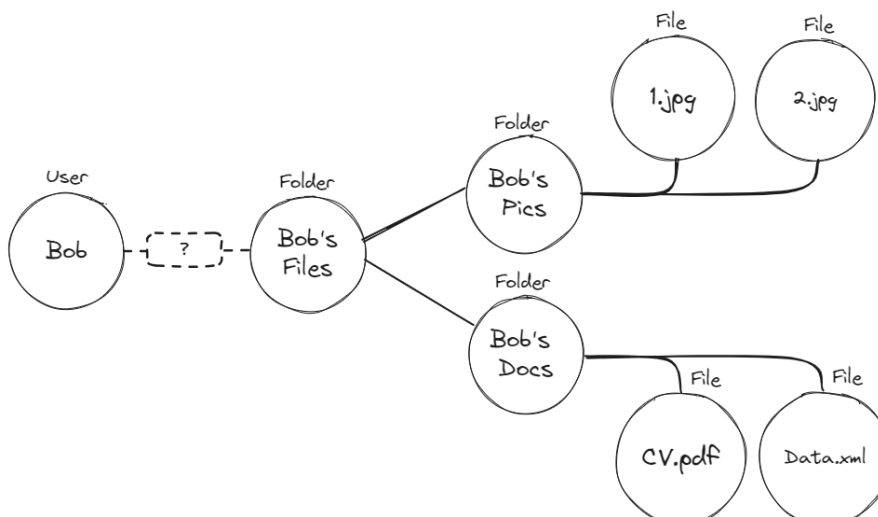


Figura 3: Modelo ReBAC.
Fonte: What is ReBAC [4]

que modelo é esse? não teve explicação anterior

2.2 Arquitetura de referência para autorização

Para que o modelo PBAM funcione, o processo de autorização é composto por quatro componentes principais: *Policy Enforcement Point* (PEP), *Policy Decision Point* (PDP), *Policy Information Point* (PIP) e *Policy Administration Point* (PAP).

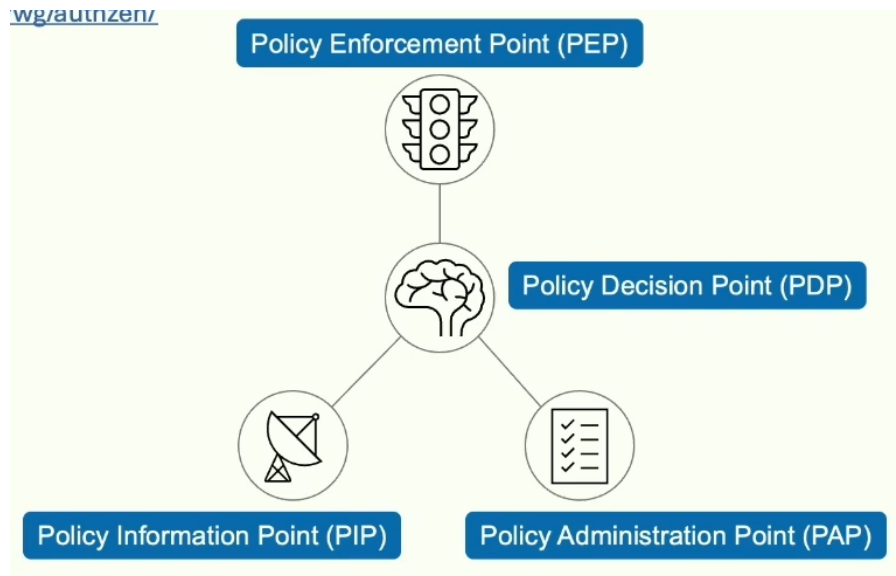


Figura 4: Modelo P*P.

Fonte: Why we need a new standard for authorization [3]

- **PEP:** Atua como um *gateway* que intercepta a solicitação do usuário e envia ao PDP.
- **PDP:** Recebe a solicitação do PEP, analisa as políticas e devolve a decisão final: *true* ou *false*. Na prática, será o OPA (será citado no próximo subcapítulo).
- **PIP:** É quem fornece os dados ou atributos necessários para o PDP avaliar a política de acesso.
- **PAP:** É onde as políticas são criadas e gerenciadas. Neste caso, o repositório em que ela está armazenada.

2.3 Gerenciamento de acesso baseado em política

A principal característica do modelo PBAM é a separação de funções. A lógica de tomada de decisão é desacoplada de dentro da aplicação principal e transferida para um serviço externo a fim de centralizar as regras de controle de acesso do sistema. Dessa forma, quando a política de acesso precisa ser atualizada, a alteração pode ser realizada em um único local, eliminando a necessidade de modificar a lógica em todos os sistemas separadamente ou realizar um novo deploy.

Com isso, a política passa a ser tratada como um código. Como exemplo, a linguagem Rego, padrão do modelo *Open Policy Agent* (OPA).

Este modelo facilita consideravelmente a auditoria e o versionamento do código. Cada mudança na política tem informações, como autor e data de alteração, que são salvas no local onde a política está armazenada, como um repositório Git.

Porém, para a sua implementação, faz-se necessário um intermediário (*middleware*). Quando uma solicitação é realizada, esse intermediário consulta o PDP OPA, que por sua vez avalia o código Rego e devolve uma resposta simples: *true* ou *false*.

seção ficou curta, sem o aprofundamento para entender OPA, etc.

Listagem 1: Exemplo de política escrita na linguagem Rego

```
// sistema: peoplefinder
// é acionado quando alguém tenta atualizar um usuário
package peoplefinder.POST.api.users._id

// arquitetura zero-trust
default allowed = false // negar por padrão
default visible = true
default enabled = true

// verifica o departamento do usuário
allowed {
    props = input.user.attributes.properties
    props.department == "Operations"
}

// caso não seja "operations", verifica se o solicitante é gerente do alvo
allowed {
    dir.is_manager_of(input.user.id, input.resource.id)
}

// só é liberado caso uma das duas condições anteriores é satisfeita
enabled {
    allowed
}
```

2.4 Validações de acesso em tempo real

????

A autorização é um problema em sistemas distribuídos, onde o grande desafio é o **controle de latência**. A etapa de login não causa problemas caso seja necessário aguardar alguns segundos para ser concluída, pois acontece uma única vez. Mas a autorização está no caminho crítico de qualquer aplicação, qualquer atraso na consulta ao PDP prejudica a usabilidade.

Para que essa validação imediata seja possível, o PDP precisa estar o mais próximo possível do PEP. Para acelerar o processo, o autorizador traz consigo todos os dados necessários em sua memória ou banco de dados local. Isso permite que a pergunta "o usuário pode acessar este recurso?" seja respondida quase instantaneamente.

Quando a política precisa ser alterada, o PDP recebe a alteração do PAP praticamente em tempo real. Além disso, todas as decisões e alterações ficam armazenadas no log do sistema central, possibilitando a auditoria posterior.

não entendi o que agrega essa seção

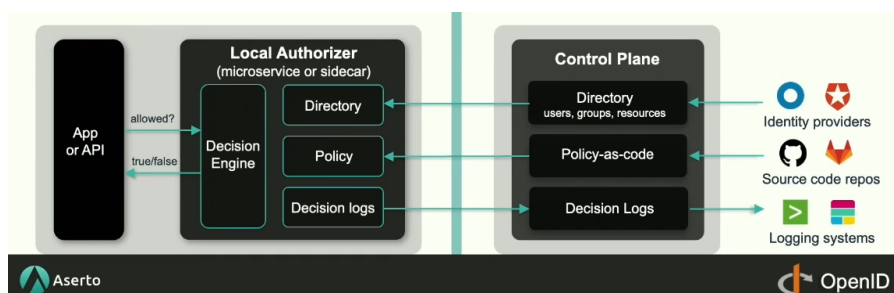


Figura 5: Exemplo de processo para validação de acesso em tempo real.

Fonte: *Why we need a new standard for authorization* [3]

3 Protocolo AuthZEN

anteriormente você chamou de modelo

Com o objetivo de alcançar todo o potencial da **arquitetura PBAM** em um sistema com diferentes tecnologias, é necessária uma linguagem comum. Com essa proposta nasce o AuthZEN, um grupo da OpenID Foundation criado com o objetivo de padronizar a autorização da mesma forma que protocolos como SAML e OIDC padronizaram

não se cria uma seção com um único parágrafo

a autenticação com o *Single Sign-On*. Ele define uma API padronizada baseada em JSON para a comunicação entre PEP e PDP.

3.1 Exemplo de solicitação de acesso

Listagem 2: Exemplo de solicitação de acesso com AuthZEN

```
{
  "subject": {
    "type": "user",
    "id": "user@example.com"
  },
  "resource": {
    "type": "account",
    "id": "123"
  },
  "action": {
    "name": "can_read",
    "properties": {
      "method": "GET"
    }
  },
  "context": {
    "time": "2026-04-30T01:22-07:00"
  }
}
```

No exemplo acima está uma demonstração de como o PEP envia a solicitação ao PDP. O código se divide em sujeito (*subject*), recurso (*resource*), ação (*action*) e contexto (*context*), que serão explicados a seguir:

O sujeito é quem está solicitando o acesso, podendo ser um ser humano, um serviço de terceiros ou até um agente de IA. O recurso é o alvo da solicitação, o que o usuário quer acessar. Para verificar se tem ou não acesso ao recurso, o usuário precisa informar uma ação. No contexto, são fornecidos dados que não são nem do usuário e nem do recurso, mas importam para a decisão, como o horário da solicitação ou o endereço IP do solicitante.

Essas informações são enviadas ao PDP, que por sua vez devolve uma resposta muito simples também em JSON: "decision": true ou "decision": false.

3.2 Defesa em profundidade

Além de unificar diversos sistemas com a mesma linguagem, o AuthZEN também permite a divisão do processo em 3 principais pontos: autenticação (*authentication*), *API gateway* e *API code*.

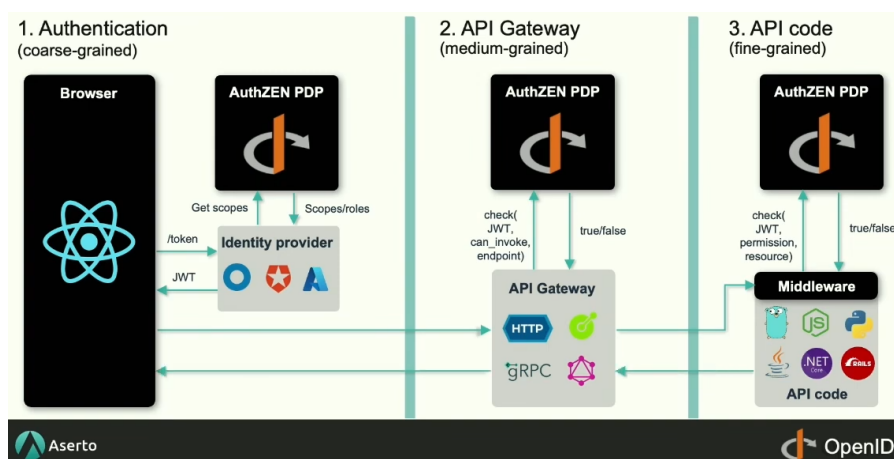


Figura 6: Exemplo de divisão do processo em etapas.

Fonte: Why we need a new standard for authorization [3]

- **Autenticação (granularidade baixa):** Nesta etapa, O provedor de identidade consulta o AuthZEN para saber quais permissões básicas devem ser adicionadas no token inicial.
- **API gateway (granularidade média):** Aqui a requisição é interceptada pelo *gateway*, que verifica com o AuthZEN se o usuário tem permissão para entrar na rota, evitando que avance sem permissão e consuma processamento desnecessário.
- **API code (granularidade alta):** Por fim, o AuthZEN é utilizado através do *middleware* para verificar se o usuário pode ou não realizar a ação solicitada.

4 Aplicação no contexto de IA Agêntica

Com a inteligência artificial agêntica, sistemas passam a tomar decisões e executar ações de forma autônoma em nome do usuário. De acordo com o guia de ameaças e mitigações da OWASP (Projeto Aberto de Segurança em Aplicações Web), essa nova possibilidade carrega consigo graves ameaças à segurança digital.

~~4.1 Velocidade e volume de requisições~~

Não se faz seções com um único parágrafo

Diferente de um ser humano, um agente de IA realiza centenas, ou até milhares, de ações em poucos segundos. Com um modelo tradicional, o volume de solicitações poderia causar lentidão ou até mesmo derrubar o sistema. Para que seja viável no contexto da IA Agêntica, o PDP precisa estar lado a lado com o sistema a fim de diminuir a latência e aprimorar o processo, condição que é satisfeita com o protocolo AuthZEN.

~~4.2 Tokens estáticos~~

Se um agente de IA receber um token estático que possibilite a personificação do usuário, ou seja, todas as permissões, e ser afetado com envenenamento de memória ou injeção de *prompts*, pode causar problemas graves no sistema em questão.

O AuthZEN contribui com a delegação dinâmica de permissões para evitar a utilização dos **stale tokens**, também permitindo que os privilégios sejam limitados estritamente ao necessário, seguindo o modelo *zero-trust*. Além disso, as ações do agente são validadas em tempo real de acordo com a política central, e caso um comportamento anômalo seja detectado, os privilégios podem ser revogados no mesmo instante em que for detectado.

~~4.3 Auditoria e rastreabilidade~~

Como citado no começo do relatório, caso a IA utilize o mesmo login e senha que o usuário humano, o sistema não consegue diferenciar quem fez o quê. Mesmo que tenha um registro de ações imutável, o autor seria o mesmo.

O AuthZEN exige o preenchimento dos campos *subject* e *context* a cada solicitação, permitindo que o sistema consiga registrar com exatidão quem é quem e quem fez o quê. Isso garante que as ações sejam registradas e auditadas de forma centralizada.

Listagem 3: Campos *subject* e *context* da solicitação em Rego

```
{
  "subject": {
    "type": "user",
    "id": "user@example.com"
  },
  "context": {
    "time": "2026-04-30T01:22-07:00"
  }
}
```

5 Conclusão

O controle de acesso precisou passar por uma profunda transformação, saindo de modelos centralizados como ACL e RBAC, que utilizam tokens estáticos, para adotar arquiteturas dinâmicas e baseadas em políticas, como o PBAM e o ReBAC, com a finalidade de atender às expectativas funcionais de um sistema moderno.

Já com a chegada da inteligência artificial agêntica, essa mudança de arquitetura se tornou uma exigência de segurança. Como os agentes tomam muitas decisões em um intervalo muito curto de tempo e podem agir em nome do usuário, depender de personificação ou conceder acesso total aumenta consideravelmente a superfície para ataques.

Em resumo, o protocolo AuthZEN atende essas solicitações. Ele garante que APIs de diferentes sistemas consigam consultar PDPs locais em tempo real. Com isso, o sistema age baseado no modelo *zero-trust*, garantindo delegação controlada de privilégios, baixa latência e rastreabilidade total das ações, aumentando consideravelmente a segurança ao utilizar IAs Agênticas.

Referências

- [1] Emerson Ribeiro de Mello. Autorização e controle de acesso, 2024. Slides de aula - Instituto Federal de Santa Catarina.
- [2] Emerson Ribeiro de Mello, Shirlei Aparecida de Chaves, Carlos Eduardo da Silva, Michelle Silva Wangham, Andrey Brito, and Marco Aurélio Amaral Henriques. *Autenticação e Autorização: antigas demandas, novos desafios e tecnologias emergentes*, pages 1–50. Sociedade Brasileira de Computação, Porto Alegre, RS, 9 2022.
- [3] Omri Gazitt. Authzen: Why we need a new standard for authorization, 2024.
- [4] Permit.io. Relationship-based access control (rbac) overview, 2024.